



Chapter 1

INTRODUCTION To EMBEDDED SYSTEMS

1



What is an Embedded System ?

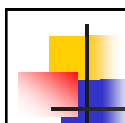
- Any electronic system that uses a computer chip, but that is not a general-purpose workstation, desktop or laptop computer.
- An embedded system is some combination of computer hardware and software, either fixed in capability or programmable, that is specifically designed for a particular function.
- An embedded system is a multi-agent system and computer system designed for specific control functions within a larger system, often with real-time computing constraints.

2



- [Peter Marwedel, TU Dortmund]
Embedded Systems = Information processing systems embedded into a larger product
- [Edward A. Lee, UC Berkeley]
Embedded Software = Software integrated with physical processes. The technical problem is managing time and concurrency in computational systems.
-

3



Where are embedded system used ?

- Everywhere
 - industrial machines
 - automobiles, trains (xe ô tô, xe lửa)
 - airplanes, space vehicles (máy bay, tàu khg gian)
 - medical equipment (thiết bị y tế)
 - video games, cameras, MP3 players, TVs
 - cell phones
 - vending machines (máy bán hàng tự động), household appliances (tb gia dụng), toys
 - ...

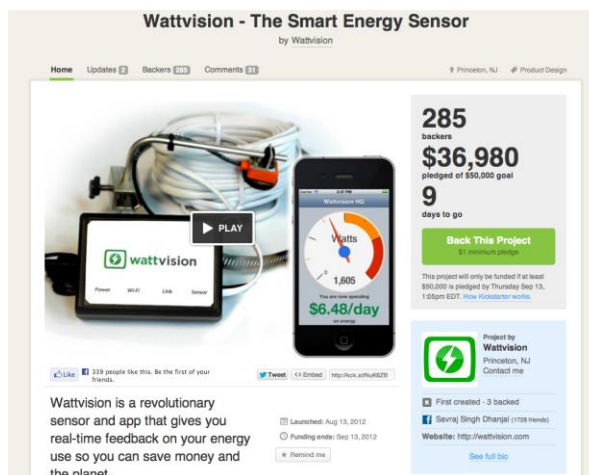
4

[illegible]

Embedded, Everywhere – WattVision on Kickstarter



Giám sát năng lượng



7

Examples

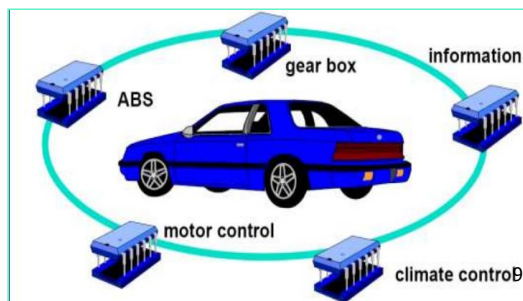
- Examples of embedded computers:
 - Engine management system in your car, etc...
 - Digital Camera
 - Mobile Phone
 - DVD player
 - TV
 - Washing Machine
 - Digital Radio
 - Heating Boiler
 - vending machine



Examples of Embedded Systems: Automotive Electronics

■ Functions by embedded processing:

- ABS: Anti-lock braking systems (HT chống bó cứng phanh)
- ESP: Electronic stability control (kiểm soát ổn định điện tử)
- Efficient automatic gearboxes (hộp số tự động)
- Theft prevention with smart keys (phòng chống trộm)
- Blind-angle alert systems (hệ thống cảnh báo)
- Airbags (túi khí)
- .. etc ...



Examples of Embedded Systems: Avionics

- Flight control systems,
- Anti-collision systems,
- Pilot information systems,
- Power supply system,
- Flap control system,
- Entertainment system,
- ...



Examples of Embedded Systems: Telecommunication

- Information systems
 - Wireless communication
 - Mobile phone
 - Wireless LAN
 - Closed systems for police, ambulance, rescue staff
- Geo-positioning system
 - Navigation
 - etc



11

Examples of Embedded Systems: Authentication Systems

- Finger print sensors
 - Airport security systems
 - Smartpen®
 - Access control



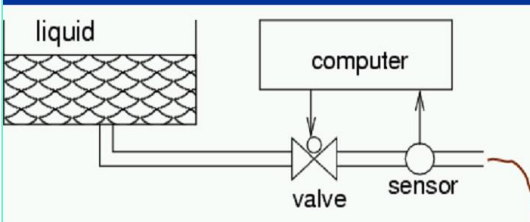
Examples of Embedded Systems: Industrial Production Systems



Chemical Installation



High-speed bonder (SMD/PCB)



13

Examples of Embedded Systems: Robotics

NASA's Mars
Sojourner Rover



Sony Aibo ERS-110
Robotic Dog

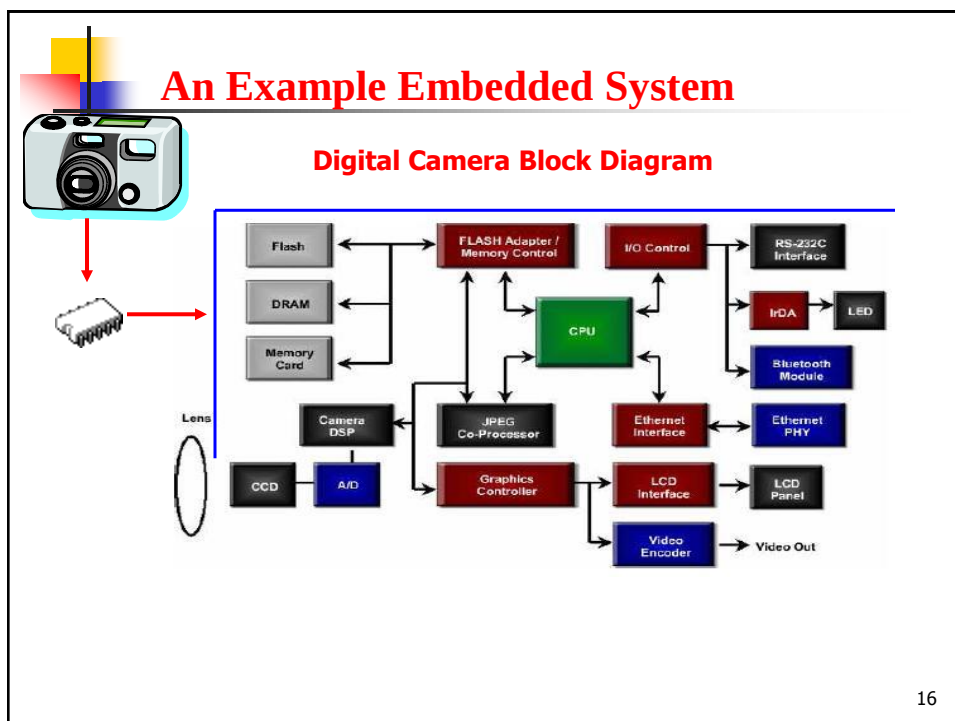


14

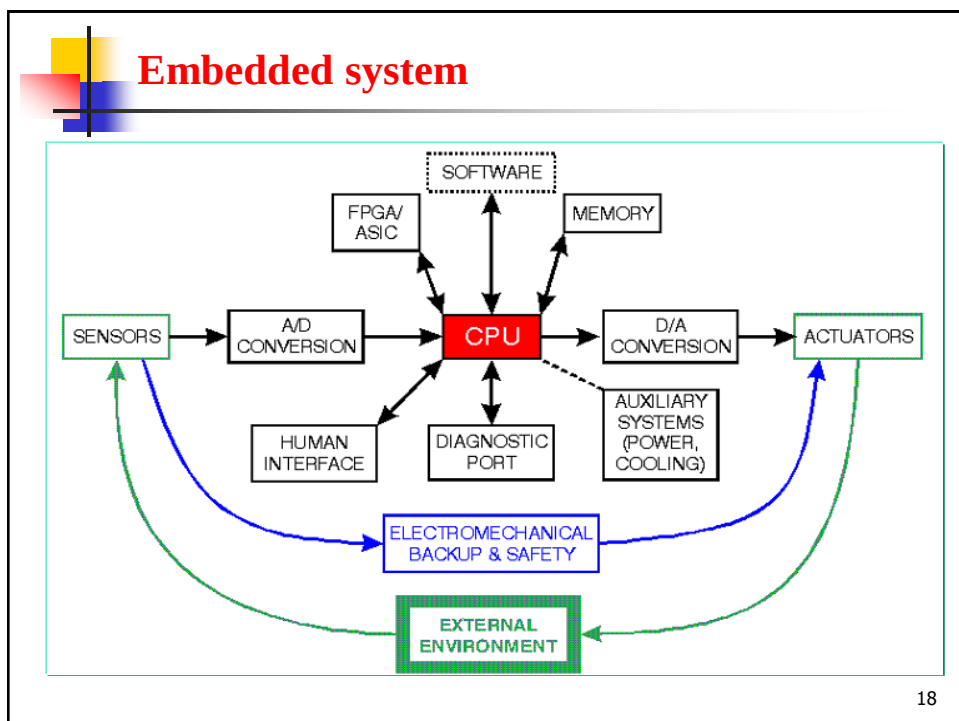
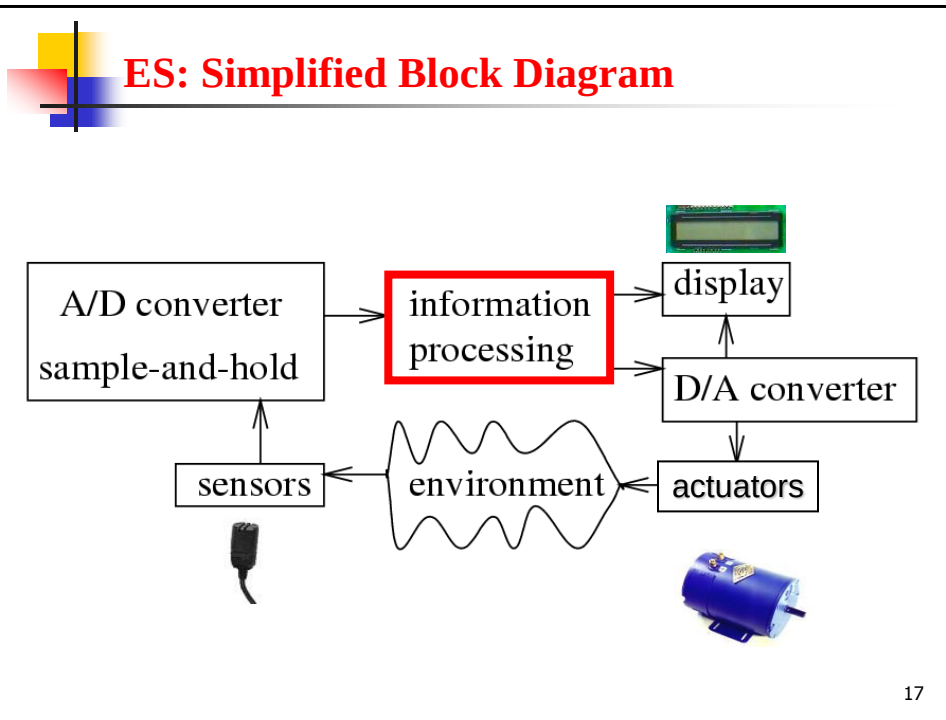
General types of Embedded systems

- General
 - similar to traditional computer systems, in a smaller package
 - PDA's
 - portable games
- Communications
 - cell phones
- Signal Processing
 - video and audio
- Control
 - real time feedback control
 - automotive (tự động)
 - aerospace (hàng không vũ trụ)
 - Appliances

15



16





Characteristics of Embedded Systems

- Must be dependable (phải đáng tin cậy)
 - Reliability: $R(t)$ = probability of a system working correctly at time t provided that it was working at $t = 0$
 - Maintainability: $M(d)$ = probability of a system working correctly d time units after error occurred
 - Availability: $A(t)$ = probability of system working at time t
 - Safety: no harm to be caused by failing system
 - Security : confidential and authentic communication

19



Characteristics of Embedded Systems

- Must be efficient
 - Energy efficient
 - Many ES are mobile systems powered by batteries
 - Customers expect long run-times from batteries but
 - Battery technology improves at a very slow rate
 - Code-size efficient (especially for systems on a chip)
 - Typically there are no hard discs or huge memories to store code
 - Run-time efficient
 - Meet time constraints with least amount of HW resources and energy – only necessary HW should be present working at as low as possible V_{dd} and f_{clk}
 - Weight efficient (especially for portable ES)
 - Cost efficient (especially for high-volume ES)

20



Characteristics of Embedded Systems

- Many ES must meet real-time constraints
 - A real-time system must react to stimuli from the controlled object (or the operator) within the time interval dictated by the environment.
 - For real-time systems, right answers arriving too late (or even too early) are wrong.
 - “A real-time constraint is called hard, if not meeting that constraint could result in a catastrophe“ [Kopetz, 1997].
 - All other time-constraints are called soft.
 - A guaranteed system response has to be explained without statistical arguments.

21



Characteristics of Embedded Systems (4)

- ES are connected to physical environment through sensors and actuators.
 - Hybrid Systems, i.e., composed of analog and digital parts
 - Typically, ES are reactive systems
 - “A reactive system is one which is in continual interaction with its environment and executes at a pace (tốc độ) determined by that environment“ [Bergé, 1995].

22



Characteristics of Embedded Systems (5)

- All ES are dedicated systems
 - Dedicated towards a certain application: Knowledge about the application at design time can be used to minimize resources and to maximize robustness
 - Dedicated user interface:
 - No mice, no large keyboards and monitors
- Not every ES has all of the above characteristics, thus : We can define the term “Embedded System” as follows: Information processing systems having most of the above characteristics are called embedded systems.

23



Comparison

- | | |
|--|---|
| <ul style="list-style-type: none"> ■ Embedded Systems <ul style="list-style-type: none"> ■ Execute few applications that are known at design-time ■ Non programmable by the end user ■ Fixed run-time requirements (additional computing power not useful) ■ Important criteria <ul style="list-style-type: none"> ■ Cost ■ Power consumption ■ Predictability | <ul style="list-style-type: none"> ■ General Purpose Systems <ul style="list-style-type: none"> ■ Execute broad class of applications ■ Programmable by the end user ■ Faster is better ■ Important criteria <ul style="list-style-type: none"> ■ Cost ■ Average speed |
|--|---|

24



Trends in Embedded Systems

- In the past Embedded Systems were called Embedded (micro-)Controllers
- They appeared typically in control dominated (chiếm ưu thế) applications:
 - Traffic lights control
 - Elevators control
 - Washing machines and dishwashers
 - ...
- They were implemented using a single μ Processor or dedicated HW (sequential circuit)
- All this is rapidly changing nowadays. How And Why?

25



Trends in Embedded Systems:

- Towards Multi-Processor Systems
- Complexity of ES is increasing, thus
 - A single μ Processor is only sufficient for some consumer products
 - Application performance demands relatively low
 - For other systems – such as cars and aircrafts – a network of processors is needed
 - Due to performance requirements
 - Due to safety requirements (a single failed component should not cause total system failure)
 - For some systems – such as mobile devices – a network of heterogeneous processors is needed
 - Due to run-time efficiency requirements
 - Due to power efficiency

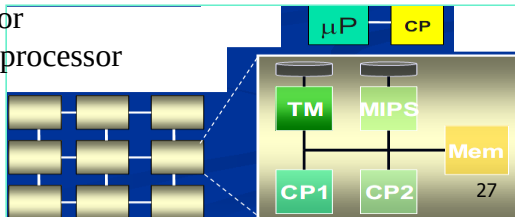
26

Trends in Embedded Systems:

- Higher Degree of Integration

Moore's Law: the number of transistors that can be placed on a chip has doubled approximately every two years

 - Microprocessor, microcontroller
 - System-on-Chip (SoC)
 - Processor + memory + I/O-units + communication structure
 - Multi-processor System on a Chip (MPSoC)
 - Processor – Co-processor
 - (Heterogeneous) Multi-processor
 - Network on Chip
 - Identical tiles
 - Scalable system



Trends in Embedded Systems:

- Software Increasing (amount and complexity)

Implementing ES in specialized HW brings lack of flexibility (changing standards) and very expensive masks, thus

 - Most of the functionality will be implemented in software
 - On the average, a human “touches” about 50 to 100 micro-processors per day
 - Average car has 15 micro-processors, luxurious one ~ 100!
 - Exponential increase in software complexity



Key recent trends

- Difficult to design
 - Planes still crash (máy bay vẫn bị rơi)
 - Car recalls... (xe hơi vẫn bị thu hồi)
- Getting even harder to design:
 - Increasing computation demands, increasing complexity
 - e.g. multimedia processing in set-top boxes, HDTV
 - Increasingly networked and distributed
 - Increasing need for flexibility
 - programmable & customizable
 - time-to-market under ever changing standards

29



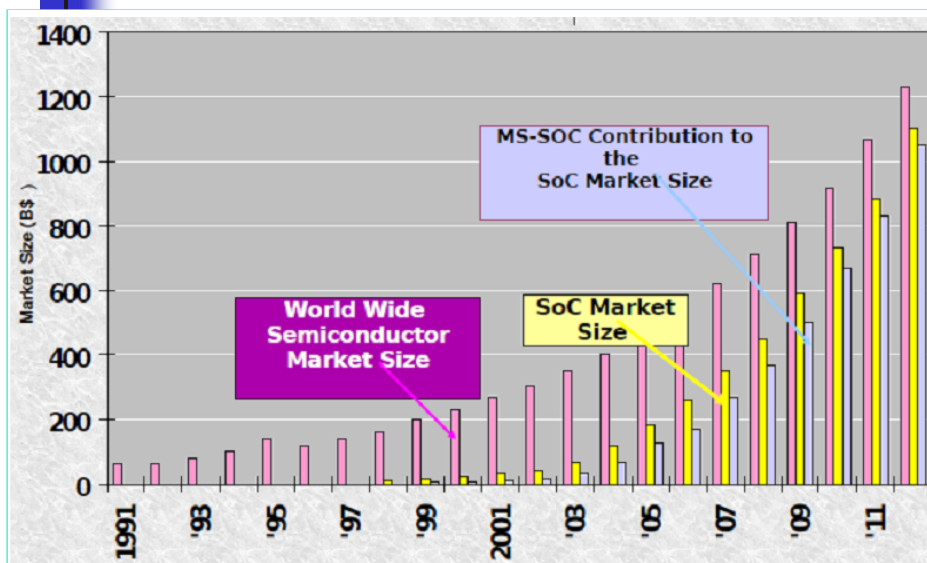
- Reaching physical limits of technology scaling
 - Power walls (and dark silicon)
 - Efficiency/optimality vs. flexibility/generality
- Technological advances
 - Higher integration: more blocks on the same chip
 - Multi-Processor System-On-Chip (MPSoC)
- Embedded systems evolve (phát triển) toward
 - System-on-Chip (SoC)
 - Cyber (liên quan tới) Physical Systems (CPS)

30

- IP reuse, platform based design, NoC vs. Bus
- HW-SW co-design
- Diversity in design methodologies, platform dependent, lack of standards, quality risks, customer confusion
- Systems are designed and built as “systems of systems”
- Opportunity and need for specialization
 - Heterogeneous multi-core / Asynchronous CMP
 - GP-GPUs

31

SoCs Market Size



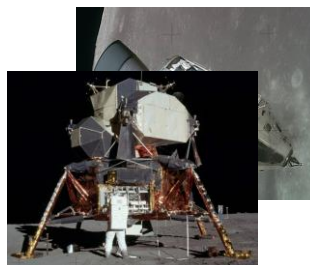
32

The Market for Embedded Processors

- Intel's Pentium Processor accounts for only about 2% of all microprocessors sold worldwide!
- Not too surprising, embedded processors within these common everyday appliances easily outnumber the single CPU in your PC or laptop.
- The worldwide market for embedded processors is worth about \$25 billion!
- Good news for us is that ARM (a UK company based in Cambridge) has the major share of this market and their processors, such as the recently released Cortex-M, is licensed to its major competitors.

History

- One of the first recognizably modern embedded systems was the Apollo Guidance Computer, developed in the early 1960s by Charles Stark Draper at the MIT Instrumentation Laboratory.
- The **Apollo Guidance Computer (AGC)** provided onboard computation and control for guidance, navigation, and control of the Command Module (CM) and Lunar Module (LM) spacecraft of the Apollo program. It is notable for having been one of the first IC based computers.



- The first computer was built using 4,100 IC's each containing a single 3 i/p XOR gate. It was considered one of the most risky elements of the mission.

35

Embedded Computers – a British success story!

The term embedded computer is used to refer to computers that are often hidden. You find them everywhere: You will find them in all kind of objects that surround us. Washing machines, mobile phones, camera's, cars, motors, sewing machines, clocks, TV's, etc..

ARM Holdings (based in Cambridge) is the biggest supplier of embedded computer 'cores'.
<http://www.arm.com/>



ARM's processors were originally conceived as CPUs for the Archimedes PC, which replaced the BBC computer.



Apple Newton PDA



The Newton platform was an early personal digital assistant and the first tablet platform developed by Apple - it used an ARM 610 RISC processor.

How ARM makes money

- ARM only creates and licenses its technology as intellectual property (IP).

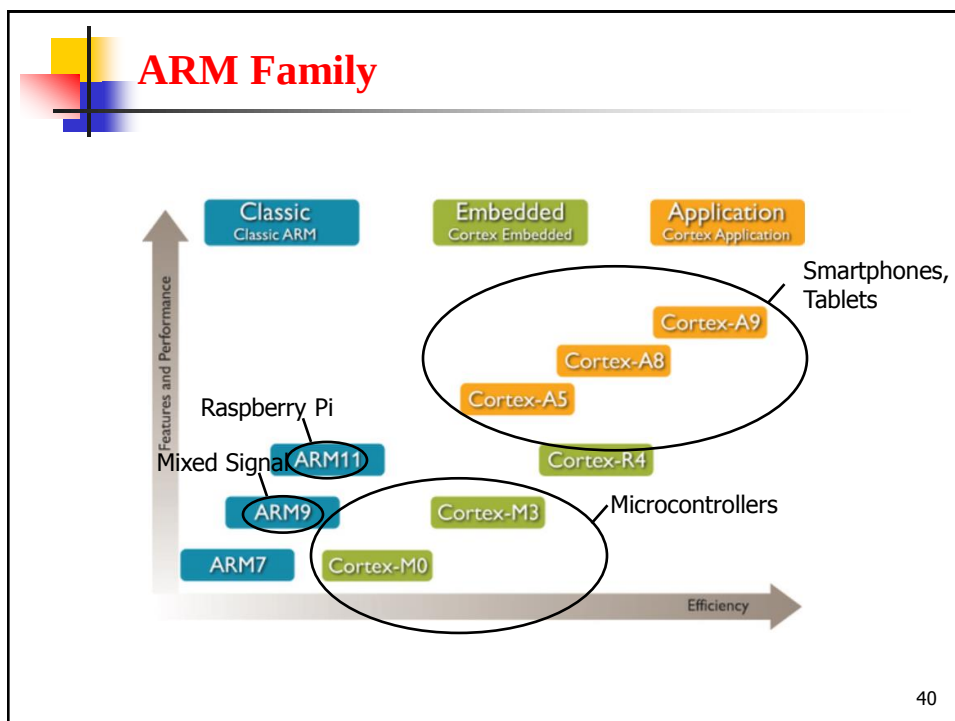


Business Model

- The partner companies who adopt ARM technology pay an up-front licence fee and also a royalty on every chip that uses the licensed design.



39





Cortex M Processors

- ARM Cortex-M processors is a global microcontroller standard, having been licensed to over 40 ARM partners including leading vendors such as Freescale, NXP Semiconductors, STMicroelectronics, Texas Instruments, and Toshiba.



41



Cortex-M Family

ARM Cortex-M	Thumb	Thumb-2	Hardware multiply	Hardware divide	Saturated math	DSP extensions	Floating-point	ARM architecture	Core architecture
Cortex-M0 ^[1]	Most	Subset	1 or 32 cycle	No	No	No	No	ARMv6-M ^[6]	Von Neumann
Cortex-M0+ ^[2]	Most	Subset	1 or 32 cycle	No	No	No	No	ARMv6-M ^[6]	Von Neumann
Cortex-M1 ^[3]	Most	Subset	3 or 33 cycle	No	No	No	No	ARMv6-M ^[6]	Von Neumann
Cortex-M3 ^[4]	Entire	Entire	1 cycle	Yes	Yes	No	No	ARMv7-M ^[7]	Harvard
Cortex-M4 ^[5]	Entire	Entire	1 cycle	Yes	Yes	Yes	Optional	ARMv7E-M ^[7]	Harvard

- Address space = 2^{32}
- Bit-Banding permits atomic access to individual bits (flags).

42



Cortex-M4

- Designed with advanced signal processing features to address digital signal control markets.
- Performance - 1.27 DMIPS/MHz

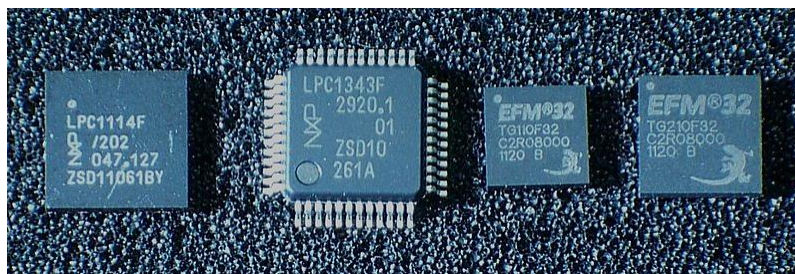


43

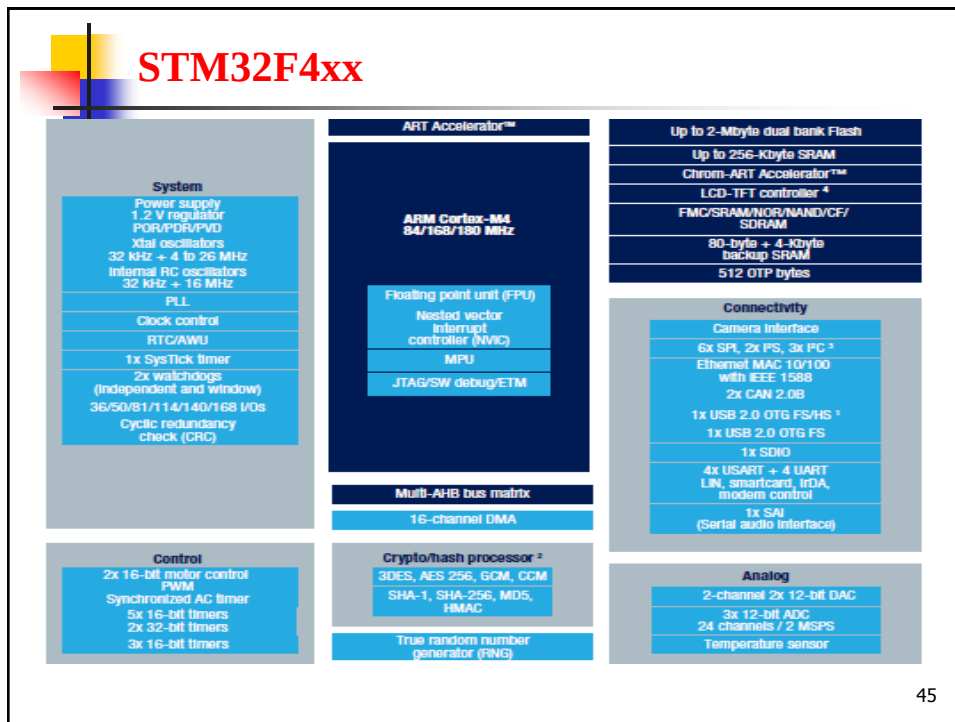


Cortex-M4 Microcontrollers

- Texas Instruments, NXP, STM, Freescale, Atmel
- ~ £7.00



44



Why study the ARM architecture ?

- Very popular in industry
- Lots of manufacturers ship ARM based products
 - What differentiates these products? Peripherals!


Actel
POWER MATTERS


ATMEL


CYPRESS
PERFORM


DUST
NETWORKS


ember


NXP


ST


TEXAS
INSTRUMENTS

46

Lots of manufacturers ship ARM products

47

ARM is *the* big player

- ARM has a huge market share
 - As of 2011 ARM has chips in about 90% of the world's mobile handsets
 - As of 2010 ARM has chips in 95% of the smartphone market, 10% of the notebook market
 - Expected to hit 40% of the notebook market in 2015.
 - Heavy use in general embedded systems.
 - Cheap to use
 - ARM appears to get an average of 8¢ per device (averaged over cheap and expensive chips).
 - Flexible
 - Spin your own designs.

48



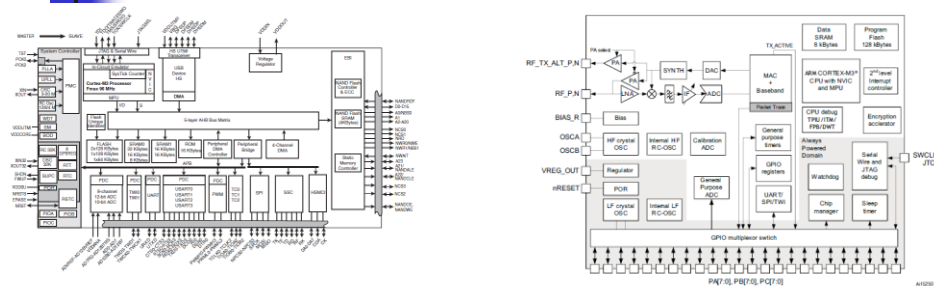
What differentiates these products from one another?

- What differentiates these products from one another?

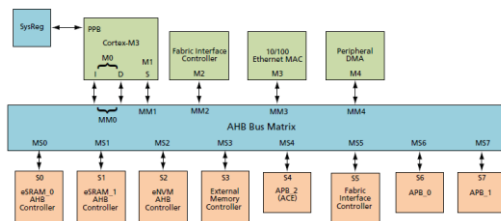
49



The difference is...



Peripherals



50

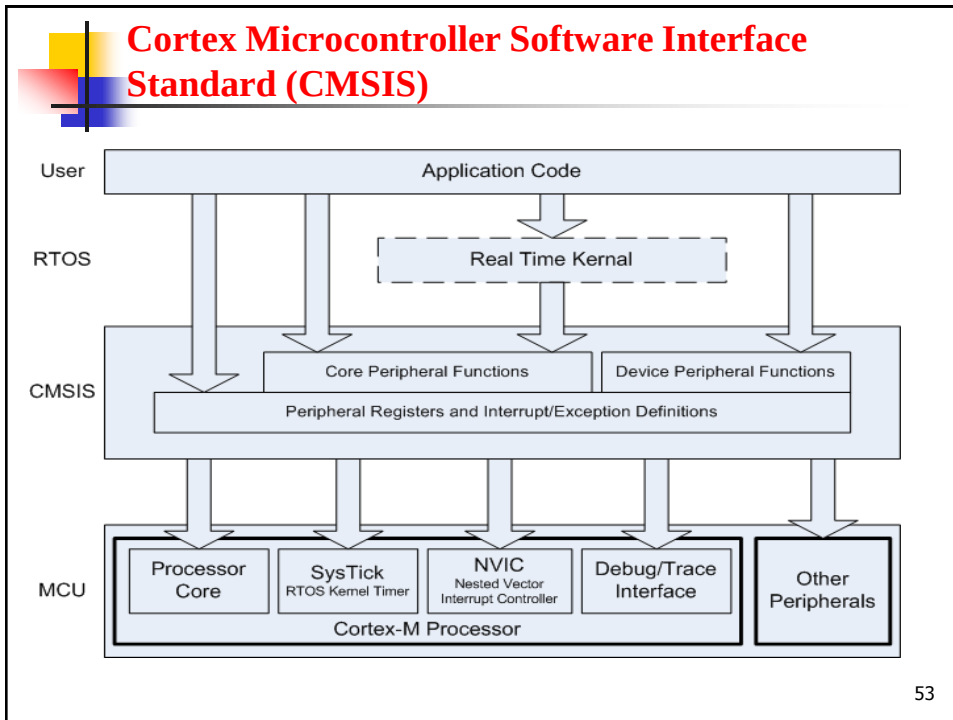


Programming Embedded

- From the programmers perspective, embedded systems show a number of significant differences from conventional “desktop” applications.
- Most desktop applications deal with a fairly predictable set of I/O devices
 - disk; graphic display, keyboard, mouse, sound card etc...
 - these are generally supported by the operating system; the application programmer doesn't need to pay much attention to them.



- Embedded systems, on the other hand, incorporate a much wider variety of I/O devices.
 - switches, various types of display, many communication channels, network ports, analogue-to-digital converters (ADC), digital-to-analogue converters (DAC), audio codec.
 - these seldom have the kind of operating system support that application programmers are accustomed to; the embedded programmer often has to deal directly with the hardware.



How to get started?

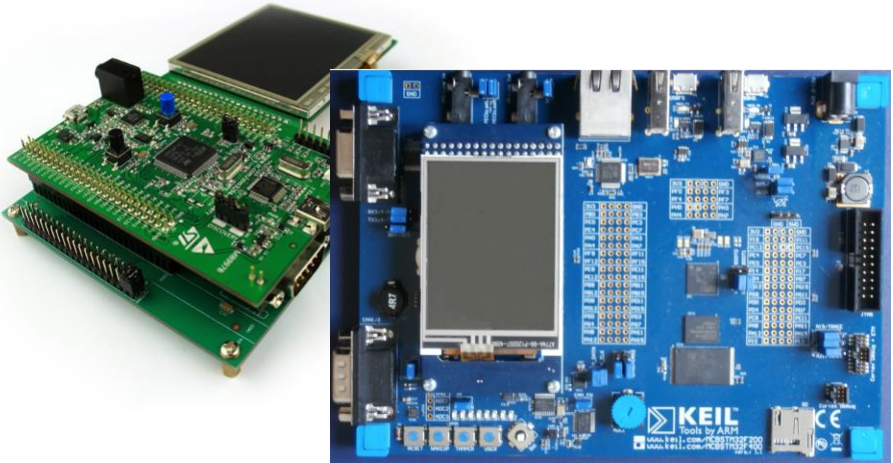
The screenshot shows the Keil MDK-ARM Version 5 - Installation page. The page includes the following sections:

- Navigation:** Products, Download, Events, Support, Videos.
- Search:** Search Keil.com.
- Quick Links:**
 - Software Packs
 - MDK Version 5
 - Legacy Support
 - Feedback
- Download:**
 - [Download MDK-Core Version 5](#)
- Installation:**
 - MDK-ARM Core Installer:** Installs the µVision IDE, Compiler, Linker, Debugger, and Simulator, and the ARM CMSIS and Keil MDK-Middleware packs.
 - PackInstaller:** Install Software Packs including Device Family Packs and Example Projects.

MDK-Lite Version 5 with 32KB Code Size limit. Use a current Product Serial Number (PSN) to activate other MDK Editions.

54

Evaluation Boards



55

Keil MCBSTM32C

MCBSTM32C
Evaluation Board and Starter Kit



The Keil MCBSTM32C Evaluation Board introduces the new STM32F107VC ARM Cortex-M3 processor-based MCU in 80-pin LQFP package, part of the STM32F107 Connectivity family of ARM Cortex-M3 processor-based devices, allowing you to create and test working programs for this advanced architecture. The MCBSTM32C has a wide range of interfaces making it a great starting point for your next Cortex-M3 project.

[Get a Quote](#)

MCBSTM32C Features

- 72MHz STM32F107VC ARM Cortex™-M3 processor-based MCU in 80-pin LQFP
- On-Chip Memory: 256KB Flash & 64KB RAM
- External Memory: 8KB I2C Flash
- Color QVGA TFT LCD with touchscreen
- 10/100 Ethernet Port
- USB 2.0 Full Speed - USB, USB-OTG, & USB Host
- 2 CAN Interfaces
- Serial Ports
- MicroSD Card Interface
- 5-position Joystick and push-button
- 3-axis Motion sensor / Accelerometer
- Analog Voltage Control for ADC Input
- Audio CODEC with Line-In/Out and Speaker
- 80 GPIO pins
- Debug Interface [Connectors](#)
 - 20 pin JTAG (0.1 inch connector)
 - 10 pin Cortex debug (0.05 inch connector)
 - 20 pin Cortex debug + ETM Trace (0.05 inch connector)



The ARM Toolchain

A toolchain is a set of programming tools that are used to create a product (typically another computer program or system of programs).

A simple software development toolchain comprises:

- text editor → compiler → linker → debugger

(assembler)



libraries

The toolchain is often wrapped in an Integrated Development Environment.



We'll use the ARM toolchain

- a commercial toolchain with user support.
- an evaluation copy can be downloaded (~mhf/embedded/)



Code Development

- Embedded systems is written in C
 - a few functions may be coded in assembler.
 - Object Oriented programming is possible, but tends to lead to less compact code.
- A modular program design paradigm is used
 - during program design, logical tasks (task sequences) become modules
 - individual modules are iteratively decomposed into smaller units, which eventually become functions.
 - modules are kept as independent as possible so they can be individually tested.



Code Development

- Modular programs comprise several source files (modules/libraries)
 - The code within each file is manageable
 - Code already implemented and tested is kept in separate files



Multi-file programs I

- Declarations and definitions
 - variables may need to be accessed by program code in a number of separate source code files, but there is only one *instance* of the variable
This leads to the separate concepts of *declaration* and *definition*
 - declaration – a specification of the name and nature of the variable
 - definition – a declaration which also reserves storage for the variable
- Novice programmers have usually written small (single source file) programs are familiar with declarations which also allocate storage.

```
float y; /* definition of external (global) variable */  
  
int main(void)  
{  
    int x; /* definition of internal variable */  
  
    /* some additional statements */  
}
```



Multi-file programs II

- Say we need to access external variables declared in one file from another file

```

/*****
 * other.c - demonstrates multi-file
 *           program.
 *****/
extern float y; /* declaration of external variable */
                /* (no storage allocated) */

void other_file_function(void)
{
    y = 6.7f;
}

```



Header files

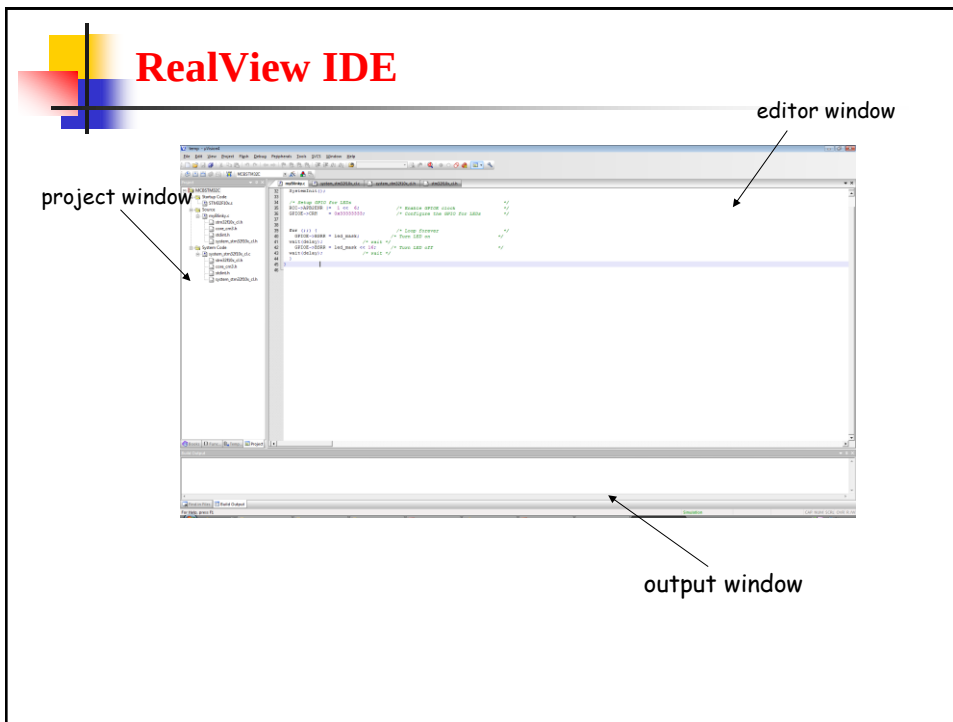
- Before a program can use functions and external variables defined in another file it must declare them .
- Although such declarations can be within the program source file this becomes tedious and error prone if there are a large number of definitions.
- A simpler alternative is to construct a header file which can be included in the program using the preprocessor **#include** directive

```

/*****
 * Header file for other.c
 *****/
extern float y;

/* function prototypes */
extern void other(void);

```



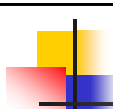
Project Window

RealView IDE allows you to gather program files into meaningful groups.

Also keeps track of included header files

Startup Code

- STM32F10x.s defines device specific initialisation code that is executed each time the processor is reset (.s file extension denotes assembly code)
- system_stm32f10x_cl.c defines further initialisation code for (some) peripherals.
- myBlinky.c defines the application code



Embedded systems and You

- As engineers, it is very likely that you will:
 - Design microprocessors and other digital circuits (e.g., ASICs, FPGAs, etc.) to be used in embedded applications
 - Develop algorithms (control, signal processing, etc.) that will be implemented on embedded microprocessors
 - Develop software (e.g., design automation – CAD – tools, RTOS, apps, etc.) for the embedded market
 - Work in application fields that involve an embedded microprocessor
 - Design sensors/actuators (e.g., MEMS devices) that may be used in embedded systems
 - Design and implement complete systems that contain embedded systems
- It is certain that you encounter embedded systems in all ⁶⁵



66